Assignment1 – Prolog

Problem 1.1 (Basic *Prolog* Functions)

Implement the *functions* listed below in *Prolog*. Note that many of them are built-in, but we ask you create your own *predicate*.

1. a *predicate reversing* a *list*
   Test case:

```
?- myReverse([1,2,3,4,2,5],R).
R = [5, 2, 4, 3, 2, 1].
```

2. a *predicate* removing multiple occurrences of *elements* in a *list*
   Test case:

```
?- removeDuplicates([1,1,1,1,2,2,3,4,1,2,7],A).
A = [1, 2, 3, 4, 7].
```

Hint: You may want to *implement* a helper *predicate* `delete(X,LS,RS)`, that removes all instances of `X` in `LS` and *returns* the result in `RS`.

---

3. a *predicate* for zipping two *lists*
   `zip` takes two *lists* and outputs a *list* of *pairs* (*represented* as 2-*element lists*) of *elements* at the same index in the two *lists*. If the *lists* do not have the same length, the zipped *list* contains only as many *pairs* as the shorter *list*.

   Create a *Prolog predicate* with 3 *arguments*: the first two are the two *lists* to zip and the third one the result. For instance:

```
?- zip([1,2,3],[4,5,6],L).
L = [[1, 4], [2, 5], [3, 6]].
```

```
?- zip([1,2],[3,4,5],L).
L = [[1, 3], [2, 4]].
```

4. a *predicate* for *computing permutations* of a *list*
   Try it out on paper first and understand why this is difficult.

   Test case:

```
?- myPermutations([1,2,3],P).
P = [1, 2, 3] ;
P = [2, 1, 3] ;
P = [2, 3, 1] ;
P = [1, 3, 2] ;
P = [3, 1, 2] ;
P = [3, 2, 1].
```

Note that there are two ways for specifying such a *function*:

(a) *return* a *list* of all *permutations*

(b) *return* a single *permutation* each time such that *Prolog* finds them one by one.

Here we are using the second way, i.e., `myPermutations(L,P)` must in particular be *true* if `P` is some *permutation* of `L`.

Hint: One possible solution is to start with a helper *predicate* `takeout(X,L,M)` that is true *iff* `M` is the result of removing the first occurrence of `X` from `L`. Or equivalently: `M` arises by adding `X` somewhere in `L`. How does this allow you to define the notion of *permutation recursively*?

Problem 1.2

1. Write *Prolog predicates* `uadd` for *addition*, `umult` for *multiplication*, and `uless` in *unary representation*.

Hint: The *number* 3 in *unary representation* is the *Prolog term* `s(s(s(z)))`, i.e. application of the arbitrary *function* `s` to an arbitrary *argument* `z` *iterated* three times.

Hint: Note that *Prolog* does not allow you to *program* (*binary*) *functions*, so you must come up with a three-place *predicate*. You should use `add(X,Y,Z)` to *mean* $X+Y=Z$ and *program* the *recursive equations* $X+0=X$ (*base case*) and $X+s(Y)=s(X+Y)$.

---

2. Write a *Prolog predicate* `ufib` that *computes* the nth *Fibonacci Number* (0, 1, 1, 2, 3, 5, 8, 13,...*add* the last two to get the next), using the *addition predicate* above.

If you have mastered *addition* and *multiplication*, feel free to try your hands on *exponentiation* as well.

Problem 1.3 (Binary Tree)

We represent the natural numbers using a nullary symbol `z` and a unary constant `s`.

A *binary tree* of (in this case) *natural numbers* is *inductively* defined as either

- an *expression* of the form `tree(n,t1,t2)` where `n` is a *natural number* (the *label* of the *node*) and `t1` and `t2` are themselves *binary trees* (the *children* of that *node*)
- or `nil` for the *empty tree*. (Normally a *tree* cannot be *empty*, but it is more convenient here to allow an *empty tree* as well.)

In particular, the *nodes* of the form `tree(n,nil,nil)` are the *leaf nodes* of the *tree*, the others are the inner *nodes*.

An example *tree* in *Prolog* would be:

```
tree(z,tree(s(s(z)),nil,nil),tree(s(s(z)),nil,nil))
```

1. Write *Prolog* predicates `isNat` and `isTree` that take one argument and succeed if that argument is a natural number or tree, respectively. Explain what happens when you run the queries `isNat(X)` or `isTree(X)`.
2. Write a *Prolog predicate* `construct` that constructs a *binary tree* out of a *list* of (distinct) *numbers* such that for every *subtree* `tree(n,t1,t2)` all *values* in `t1` are smaller than `n` and all *values* in `t2` are larger than `n`.

   Note that there are usually multiple such *trees* for every *list*. One example is:

   ```
   ?- construct([s(s(s(z))),s(s(z)),s(s(s(s(z)))),s(z),s(s(s(s(z))))],T).
   T = tree(s(s(s(z))), tree(s(s(z)), tree(s(z), nil, nil), nil),
       tree(s(s(s(s(z)))), nil, tree(s(s(s(s(z)))), nil, nil))).
   ?- isTree(T).
   true
   ```

3. Write *Prolog predicates* `count_nodes` and `count_leaves` that take a *binary tree* and *return* the number of *nodes* and *leaves*, respectively.
4. A *binary tree* is symmetric if it is its own mirror image, i.e., all *nodes* have left and right *child* switched. Write a *Prolog predicate* `symmetric` that checks whether a *binary tree* is symmetric.